



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

School of Engineering

Department of Mechanical & Manufacturing Engineering

Interim Report

In Partnership with ElectroRoute



Finn O'Connor

22336740

May 5, 2026

4E4 Industrial Internship Project

Contents

1	Introduction and Specific Technical Objective	2
1.1	Company Background	2
1.2	Internship Context and Role	2
1.3	Learning Outcomes	2
1.4	Technical Objectives	3
2	Background Theory and Literature Review	3
2.1	Electricity Grid and Market Fundamentals	3
2.2	The Integrated Single Electricity Market (I-SEM)	4
2.3	Renewable Energy Trading Considerations	5
2.4	Interconnectors and Market Coupling	6
2.5	Key Performance Metrics	6
3	Description of Technical Work	6
3.1	Development of Trading KPI Script	6
3.2	Solar Imbalance Price Trading Strategy	19
3.3	Paper Trading	21
3.4	Weekend Trading Look-Aheads	22
3.5	Data Scientist in Python	26
4	Conclusions	26
5	Plan for Future Work	27

1 Introduction and Specific Technical Objective

1.1 Company Background

ElectroRoute is an energy trading company headquartered in Dublin, Ireland. Founded in 2011, the company specialises in optimising the value of renewable energy assets by trading their output across European wholesale electricity markets. ElectroRoute manages a diverse portfolio of wind, solar, and battery storage assets, providing services including market access, trading, forecasting, and route-to-market solutions for asset owners. The company operates a 24/7 shift trading desk and a prop desk, and leverages proprietary technology platforms to maximise returns while minimising exposure to balancing costs and market volatility.

As a participant in the Integrated Single Electricity Market (I-SEM) and GB Market, ElectroRoute trades on behalf of renewable generators who face inherent uncertainty in their output. The company's core value proposition lies in its ability to forecast generation, execute trades across multiple market timeframes (Day-Ahead, Intraday Auctions, Intraday Continuous), and manage the financial risks associated with energy prices. This requires sophisticated data infrastructure, analytical tools, and trading expertise.

1.2 Internship Context and Role

I am currently undertaking an eight-month internship within ElectroRoute's Shift Trading Team, working across trading operations, data analysis, and tool development. The team operates on a rotating shift pattern to provide continuous market coverage, executing trades across all I-SEM timeframes and monitoring portfolio performance in real-time.

My role spans both operational and analytical responsibilities. On the operational side, I have gained hands-on experience with trade execution, market monitoring, and responding to changing forecasts and market conditions. On the analytical side, I work with large datasets from internal databases and external APIs, performing quantitative analysis and building tools to support trading decisions.

The internship has provided exposure to the full trading workflow: from receiving generation forecasts and market data, through trade execution in the DAM and IDAs, to settlement and performance analysis. This end-to-end perspective has deepened my understanding of how renewable energy is commercialised and the challenges involved in trading variable generation.

1.3 Learning Outcomes

So far through this internship, I have developed skills and knowledge across several areas:

- **Electricity market fundamentals:** Understanding of market structures, price formation, settlement processes, and the regulatory framework governing I-SEM and European electricity markets.
- **Renewable energy trading:** Practical experience with the challenges of trading variable generation, including forecast uncertainty, imbalance exposure, and the trade-offs between different market timeframes.
- **Data analysis and programming:** Proficiency in Python (pandas, NumPy, Plotly), SQL, and Excel for data extraction, manipulation, analysis, and visualisation.
- **Tool development:** Experience building analytical tools that are used by the trading desk, requiring consideration of usability, performance, and maintainability.
- **Commercial awareness:** Insight into how trading decisions impact financial performance, and how quantitative analysis informs strategy development.

1.4 Technical Objectives

The primary technical focus of this internship has been the development of analytical tools to evaluate and improve trading performance. The overarching objectives are:

1. **Performance analysis:** Develop tools to quantify trading performance, enabling the desk to assess how well the portfolio is being managed relative to benchmarks and historical trends.
2. **Data infrastructure:** Work with internal databases and APIs to extract, process, and analyse settlement data, market prices, forecasts, and trading positions.
3. **Visualisation and reporting:** Create interactive visualisations that communicate insights clearly and support data-driven decision-making.
4. **Process improvement:** Identify inefficiencies in existing workflows and implement improvements, such as consolidating separate analysis scripts and optimising data retrieval.

So far, my main deliverable was a comprehensive KPI analysis script that consolidates multiple data sources, calculates key trading metrics, and presents results through interactive charts. This tool is used by the trading desk to review performance across different asset groups and time periods.

2 Background Theory and Literature Review

This section provides an overview of some of the fundamental concepts, market structures, and technical foundations relevant to the work undertaken during my internship. Understanding these basics is essential for understanding the trading analysis tools and KPI scripts developed.

2.1 Electricity Grid and Market Fundamentals

2.1.1 The Nature of Electrical Power

Electricity possesses unique physical characteristics that distinguish it from other traded commodities. Most notably, electrical power cannot be economically stored at scale, necessitating continuous real-time balancing between generation and consumption. The amount of electricity fed into the grid must equal the amount consumed at any point in time—this fundamental constraint shapes the entire structure of electricity markets.

When supply and demand are not equal, the system frequency deviates from the nominal 50 Hz:

- **Excess generation** (supply > demand): frequency rises above 50 Hz, risking generator trips
- **Generation shortage** (supply < demand): frequency falls below 50 Hz, triggering automatic load shedding at approximately 49 Hz

To prevent imbalances, the Transmission System Operator (TSO) employs several mechanisms: regulating power stations using flexible plants (e.g., CCGTs), demand response from industrial customers, interconnector flows to neighbouring grids, and as a last resort, load shedding.

2.1.2 Energy Exchange and Price Formation

Energy exchanges bring together supply and demand through a structured bidding process. Market participants submit **bids** (willingness to buy at a maximum price) and **offers** (willingness to sell at a minimum price). The exchange aggregates these into supply and demand curves, with the intersection determining the **Market Clearing Price (MCP)** and **Market Clearing Volume (MCV)**.

Orders are classified as:

- **In the money:** Bids $\geq$ MCP or Offers $\leq$ MCP (cleared in the auction)
- **Out of the money:** Bids $<$ MCP or Offers $>$ MCP (not cleared)

2.1.3 Settlement Periods and Imbalances

Although electricity flows continuously, trading and settlement occur in discrete chunks called **Settlement Periods** (30 minutes in I-SEM). For each period, participants contract volumes in advance, but actual generation or consumption may differ due to forecast errors, plant issues, or transmission problems.

Differences between contracted (traded) volumes and metered volumes are called **imbalances**, settled at the **Imbalance Price** (P_{IMB}):

- Generator produces more than contracted $\rightarrow$ sells excess to grid at P_{IMB}
- Generator produces less than contracted $\rightarrow$ buys shortfall from grid at P_{IMB}
- Supplier uses more than contracted $\rightarrow$ buys excess from grid at P_{IMB}
- Supplier uses less than contracted $\rightarrow$ sells surplus to grid at P_{IMB}

2.2 The Integrated Single Electricity Market (I-SEM)

2.2.1 Market Structure Overview

The Integrated Single Electricity Market (I-SEM) is the wholesale electricity market serving the Republic of Ireland and Northern Ireland, launched in October 2018. The market comprises several sequential timeframes, each serving different purposes:

Market	Timing	Purpose
Day-Ahead (DAM)	Auction at 11:00 D-1	Primary market for establishing physical positions
Intraday (IDM)	IDA1, IDA2, IDA3 + continuous	Adjust positions as forecasts improve
Balancing (BM)	Real-time	TSO balances system; sets imbalance price

The I-SEM trading day runs from 23:00 GMT D-1 to 23:00 GMT D (aligned with midnight CET).

2.2.2 Day-Ahead Market

The DAM is a daily auction at 11:00 GMT where participants trade 24 hourly periods for the following day. It offers the highest liquidity and is the primary mechanism for establishing positions. Key characteristics include:

- Auction-based clearing with uniform marginal pricing
- Results published from 11:45
- SEMOpx acts as central counterparty (participants trade with SEMOpx, not directly with each other)

Order types include **linear orders** (price-quantity pairs for specific hours) and **complex orders** with additional conditions such as minimum income requirements or load gradient constraints (ramp rate limits).

2.2.3 Intraday Markets

The Intraday Market allows position adjustments closer to real-time through three auctions and a continuous market:

	Gate Closure	Periods Covered	Coupling
IDA1	17:30 D-1	48×30 min	SEM-GB
IDA2	08:00 D	24×30 min	SEM-GB
IDA3	14:00 D	12×30 min	Local
IDC	1 hour before delivery	48×30 min	Local

Liquidity decreases through successive auctions, creating a trade-off between forecast accuracy (which improves closer to delivery) and market depth.

2.2.4 Balancing Market and Imbalance Settlement

The Balancing Market is operated by the TSO to resolve differences between contracted positions and actual generation/demand in real-time. The imbalance price can be significantly higher or lower than day-ahead prices depending on system conditions, creating financial risk for participants with forecast errors.

A participant's **energy position** is the accumulated volume of all ex-ante trades plus any TSO balancing actions. After delivery, metered generation (Q_M) adjusted for transmission losses (LF) gives the loss-factored metered quantity (Q_{MLF}), which is compared against the ex-ante quantity (Q_{EX}) to determine imbalance volumes.

2.3 Renewable Energy Trading Considerations

2.3.1 Generator Classifications

Generators in I-SEM are classified by their dispatch characteristics:

- **Dispatchable units:** Can follow MW set-points from the TSO (e.g., CCGT plants)
- **Non-dispatchable but controllable:** Cannot ramp to targets but can limit output (e.g., commissioned wind farms)
- **Priority dispatch units:** Renewables and high-efficiency plants that the TSO must accommodate first, subject to system security

2.3.2 Curtailment and Constraints

The TSO may instruct renewable generators to reduce output for two distinct reasons:

- **Curtailment:** System-wide issue, typically when System Non-Synchronous Penetration (SNSP) reaches its limit
- **Constraints:** Local network issue, such as transmission line or voltage limits in a specific area

Compensation for dispatch-down depends on grid access rights: **firm access** generators receive compensation for lost volume, while **non-firm access** generators bear the loss themselves.

2.3.3 Balancing Cost Minimisation

The primary objective of renewable energy trading is to minimise balancing costs—the financial impact of deviations between sold positions and actual generation. The balancing cost for a given period can be expressed as:

$$C_{bal} = P_{client} - R_{net} \quad (1)$$

where P_{client} is the payment owed to the asset owner and R_{net} is the net revenue from all market activities:

$$R_{net} = R_{DA} + R_{IDA1} + R_{IDA2} + R_{IDA3} + R_{IDC} + R_{IMB} \quad (2)$$

The imbalance component is calculated as:

$$R_{IMB} = (Q_{MLF} - Q_{EX}) \times P_{IMB} \quad (3)$$

2.4 Interconnectors and Market Coupling

Interconnectors are high-voltage cables connecting electricity systems of neighbouring regions, enabling cross-border trade. Ireland's interconnectors (EWIC, Moyle, and the forthcoming Celtic Interconnector) allow imports when Ireland is short and exports during surplus periods.

In **coupled markets**, buy and sell orders from different bidding zones are cleared together, with cross-border flows optimised automatically. When there is no congestion, prices converge across zones. When interconnector capacity is constrained, prices diverge, generating **congestion rent** for interconnector owners.

Following Brexit, I-SEM is no longer coupled with continental European markets but maintains cross-border trading with GB. The Celtic Interconnector will restore access to the European coupled market upon completion.

2.5 Key Performance Metrics

Effective trading desk management requires quantitative metrics to assess performance. Key indicators analysed during this internship include:

- **Balancing Cost per MWh (€/MWh)**: Total balancing costs divided by metered generation
- **Imbalance Position**: Difference between sold volume and metered generation ($Q_{EX} - Q_{MLF}$)
- **Intraday Auction Gain**: Value added by trading in IDAs compared to settling at imbalance prices
- **Dispatch-Down Percentage**: Proportion of available generation lost to curtailment and constraints
- **Capacity Factor**: Ratio of average production to maximum capability

These metrics enable the trading desk to identify patterns, benchmark performance against counterfactual strategies, and continuously refine trading approaches.

3 Description of Technical Work

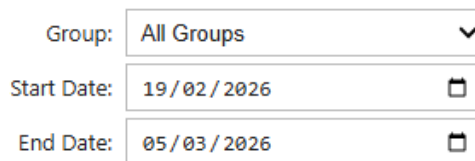
3.1 Development of Trading KPI Script

One of the most substantial tasks during my internship has been the development of a comprehensive KPI (Key Performance Indicator) Python script to analyse the trading performance of ElectroRoute's asset

portfolio. The script retrieves data from internal SQL databases, performs financial calculations, and presents insights through interactive Plotly charts. This enables the trading desk to evaluate historical performance, identify effective strategies, and highlight areas for improvement. The complete Jupyter Notebook contains over 4,700 lines of code; a representative extract of the core processing logic is included in Appendix ??.

Timeframe-Based Aggregation

Previously, two separate KPI scripts existed—one for year-to-date (YTD) analysis and one for month-to-date (MTD). I consolidated these into a single unified script with automatic aggregation based on the selected timeframe. When the user runs the first cell, an interactive widget prompts them to select the asset group(s) and analysis date range. If the selected range exceeds 62 days (approximately two months), the script automatically aggregates data to monthly periods and converts line charts to bar charts where appropriate. For shorter timeframes, daily granularity is retained.



The figure shows an interactive widget with three input fields. The first field is labeled 'Group:' and has a dropdown menu with 'All Groups' selected. The second field is labeled 'Start Date:' and has the date '19/02/2026' entered. The third field is labeled 'End Date:' and has the date '05/03/2026' entered. Each date field has a calendar icon to its right.

Figure 1: Interactive widget for selecting asset groups and analysis date range.

SQL Query Optimisation

The original script suffered from slow query times when retrieving data from SQL databases. I implemented several optimisations and added progress bars to provide visibility into which queries were causing delays. This diagnostic approach identified that fetching position reports for the REFIT wind farm group was the primary bottleneck. The issue was passed to the data engineering team, who are currently developing a more efficient data retrieval system.

```
=====
Running analysis with:
Groups: ['DAM', 'RFIT', 'SOL']
Start: 2026-02-19
End: 2026-03-05
=====
Timeframe: 14 days (<=62 days)
Aggregation: Daily
=====

Fetching common data (prices, SNRP)...
Common Data: 100% ██████████ 2/2
Common data loaded successfully!

Processing Groups: 100% ██████████ 3/3 [06:39<00:00]

All groups processed successfully!
```

Figure 2: Progress bar output showing data retrieval and processing status for each asset group.

Core Processing Logic

The main processing block executes a data pipeline for each asset group, combining market data, settlement information, and forecasts to calculate trading performance metrics. The key stages are outlined below.

1. Configuration and Date Handling:

```
# Validate date inputs
if start > end:
```

```

        raise ValueError("Start date must be before end date!")

# Determine aggregation frequency based on analysis timeframe
timeframe_days = (pd.to_datetime(end) - pd.to_datetime(start)).days
TWO_MONTHS_DAYS = 62
if timeframe_days > TWO_MONTHS_DAYS:
    AGG_FREQ, AGG_LABEL = "MS", "Monthly" # Month-start frequency
    adj_start = pd.to_datetime(start).replace(day=1) # Align to month
    ↪ boundary
else:
    AGG_FREQ, AGG_LABEL = "D", "Daily"

```

2. Data Retrieval:

```

# Fetch I-SEM market prices (half-hourly resolution)
df_Prices = pows.all_isem_prices(start, end) # Columns: PDA, PIDA1, PIDA2,
↪ PIDA3, PIMB
df_Prices["PIMB"].fillna(df_Prices["PDA"], inplace=True) # Handle missing
↪ imbalance prices

# System data
nsp_df = tops.irl_SNSP(start, end) # System Non-Synchronous Penetration (%)

# Per-group data retrieval via SQL
unit_sql = f'''SELECT isem_unit_id, project FROM
↪ unit_mapping.tb_isem_unit_mapping
        WHERE mgmt_group = "{grp}" AND isem_unit_id LIKE "%U_%"'
df_group = pd.DataFrame(dbConnect.select(unit_sql, "ISEM-Prod"),
↪ columns=["Unit_ID", "Name"])

# Installed capacity time series (handles contract start/end dates)
df_capacity = pd.DataFrame(index=pd.date_range(df_cap.Contract_Start.min(),
        df_cap.Contract_End.max()))
df_capacity[grp] = df_capacity.apply(
    lambda x: df_cap.loc[(df_cap.Contract_Start <= x.name) &
        (x.name <= df_cap.Contract_End), [grp]].sum(),
    ↪ axis=1)

# Generation forecasts from each auction gate
DA_fcast = fors.unit_forecast_DA(grp, "E&M", start, end)
IDA1_fcast = fors.unit_forecast_IDA1(grp, "E&M", start, end)
IDA2_fcast = fors.unit_forecast_IDA2(grp, "E&M", start, end)
IDA3_fcast = fors.unit_forecast_IDA3(grp, "E&M", start, end)
df_fcast = pd.concat([DA_fcast, IDA1_fcast, IDA2_fcast, IDA3_fcast], axis=1)

# Trading positions (MW sold/bought per auction)
df_positions = fors.position_dataframe(grp, start, end) # Columns: DA_MW,
↪ IDA1_MW, etc.

# Settlement data
df_SR = fors.irl_settlement_reports_v2(start, end, group=grp) # Volumes: QM,
↪ QEX, QAB, etc.
df_SS = fors.irl_settlement_statements_v2(start, end, group=grp) # Financials:
↪ DA_EUR, CIMB, etc.

```

3. Data Consolidation and Preprocessing:

```

# Merge all data sources on datetime index

```

```

df = pd.concat([df_SS, df_SR, df_fcast, df_Prices, df_positions, snsp_df],
    ↪ axis=1)
df = df.loc[(df.index >= pd.to_datetime(start)) & (df.index <=
    ↪ latest_settlement)]

# Apply loss factors to forecasts (grid losses adjustment)
df_LF = df_SRx.loc[df_SRx["determinant"] == "LF"]
df["LF"] = df_LF.groupby(["time_IST"])["value"].mean().ffill()
df["DA_Forecast"] = df["DA_Forecast"] * df["LF"]

# Ensure all required columns exist (fill missing with zeros)
required_cols = ("QAB", "QABCURL", "QABNF", "QEX", "QM", "CCURL", "CDISCOUNT",
    ↪ "CIMB")
for col in required_cols:
    if col not in df.columns:
        df[col] = 0

```

4. Core Financial Calculations:

```

# Total exchange auction income (sum of all auction revenues)
df["ExA_Income"] = (df["DA_EUR"] + df["IDA1_EUR"] + df["IDA2_EUR"]
    + df["IDA3_EUR"] + df["IDC_EUR"]).fillna(0)

# Balancing market charges (group-specific)
bmcharges = {"RFIT": df["CDIFFPIMB"],
    "DAM": ERG_charges, # Energia charges for dispatchable assets
    "COR": df["CCURL"] + df["CDISCOUNT"]}.get(grp, 0)

# Imbalance settlement income
df["IMB_Income"] = df["CIMB"] + bmcharges

# Net income from all market activities
df["Net_Income"] = df["ExA_Income"] + df["IMB_Income"]

# Client payment calculation (revenue owed to asset owner)
# REFIT assets use blended price: min(PDA, 0.8*PDA + 0.2*PIMB)
df["RFIT_Price"] = np.minimum(df["PDA"], 0.8 * df["PDA"] + 0.2 * df["PIMB"])
df["Client_Payment"] = df["QMLF"] * (df["RFIT_Price"] if grp in ["RFIT",
    ↪ "COR"] else df["PDA"])

# Balancing cost = Client Payment - Net Income (positive = cost to desk)
df["Cost"] = df["Client_Payment"] - df["Net_Income"]

```

5. Intraday Auction Performance Analysis:

```

# Calculate gain/loss from each intraday auction relative to imbalance
    ↪ settlement
# Gain = Auction Revenue + (Volume * Imbalance Price) [what we avoided
    ↪ paying/received]
# Division by 2 converts MW to MWh for 30-minute periods
df["IDA1_Gain"] = df["IDA1_EUR"] + df["IDA1_MW"] * df["PIMB"].fillna(0) / 2
df["IDA2_Gain"] = df["IDA2_EUR"] + df["IDA2_MW"] * df["PIMB"].fillna(0) / 2
df["IDA3_Gain"] = df["IDA3_EUR"] + df["IDA3_MW"] * df["PIMB"].fillna(0) / 2
df["IDC_Gain"] = df["IDC_EUR"] + df["IDC_MW"] * df["PIMB"].fillna(0) / 2

# Cumulative intraday trading value-add
df["Net_Gain"] = df["IDA1_Gain"] + df["IDA2_Gain"] + df["IDA3_Gain"] +
    ↪ df["IDC_Gain"]
df["Cumulative_IDA_Gain"] = df["Net_Gain"].cumsum()

```

6. Forecast Scenario Comparison:

```
# Simulate costs under different DA trading strategies (90%, 100%, 110% of
↪ forecast)
for pct, label in [(1.0, "100"), (0.9, "90"), (1.1, "110")]:
    df[f"{label}_DA_forecast"] = df["DA_Forecast"] * pct
    forecast_vol = df[f"{label}_DA_forecast"] / 2 # Convert to MWh

    # Simulated cost: Client_Payment - (DA_Revenue + Imbalance_Settlement +
    ↪ Charges)
    # Imbalance volume = Metered - Sold at DA
    imb_vol = df["QMLF"] - forecast_vol
    df[f"{label}_Forecast_Cost"] = (df["Client_Payment"]
                                   - (forecast_vol * df["PDA"] + imb_vol *
                                   ↪ df["PIMB"] + bmcharges))
```

7. Aggregation and Results Storage:

```
# Aggregate metrics by selected frequency
Agg_Cost = df["Cost"].resample(AGG_FREQ).sum()
Agg_Prod = df["QMLF"].resample(AGG_FREQ).sum()
Agg_Weighted_Cost = Agg_Cost / Agg_Prod # EUR/MWh per period

# Cumulative metrics for trend analysis
Agg_Cost_cumsum = Agg_Cost.cumsum()
Agg_Prod_cumsum = Agg_Prod.cumsum()
Rolling_cost = Agg_Cost_cumsum / Agg_Prod_cumsum # Running EUR/MWh

# Total period metrics
Total_Cost, Total_Prod = df["Cost"].sum(), df["QMLF"].sum()
Cost_per_MWh = Total_Cost / Total_Prod

# Store all results in dictionary for downstream plotting
results[grp] = {
    "df": df, "start": grp_start, "end": grp_end,
    "agg_freq": AGG_FREQ, "agg_label": AGG_LABEL,
    "Total_Cost": Total_Cost, "Total_Prod": Total_Prod, "Cost_per_MWh":
    ↪ Cost_per_MWh,
    "Agg_Cost": Agg_Cost, "Agg_Prod": Agg_Prod, "Agg_Weighted_Cost":
    ↪ Agg_Weighted_Cost,
    "Rolling_cost": Rolling_cost, "Agg_Cost_cumsum": Agg_Cost_cumsum,
    # ... additional scenario results
}
```

The script processes all selected asset groups sequentially, with progress tracking via `tqdm` progress bars. Upon completion, a summary table displays key metrics (total production, total cost, cost per MWh) for each group, enabling rapid comparison of portfolio performance.

New KPIs

I created many new KPIs for the team to analyse, enabling more granular insight into trading performance and market conditions. The data was displayed using interactive Plotly charts, allowing users to hover over data points, zoom into specific time periods, and toggle individual traces on and off. Below are some examples of these plots — please note that some figures have been redacted due to proprietary company information.

Market Prices Overview

This KPI provides a consolidated view of prices across all I-SEM auction markets: Day-Ahead (DA), Intraday Auction 1 (IDA1), Intraday Auction 2 (IDA2), Intraday Auction 3 (IDA3), and the Imbalance Market. Visualising these prices together allows traders to identify price trends, spot arbitrage opportunities between auctions, and understand the typical price evolution from DA through to real-time settlement. This is particularly useful for evaluating whether holding volume for later auctions would have been profitable.

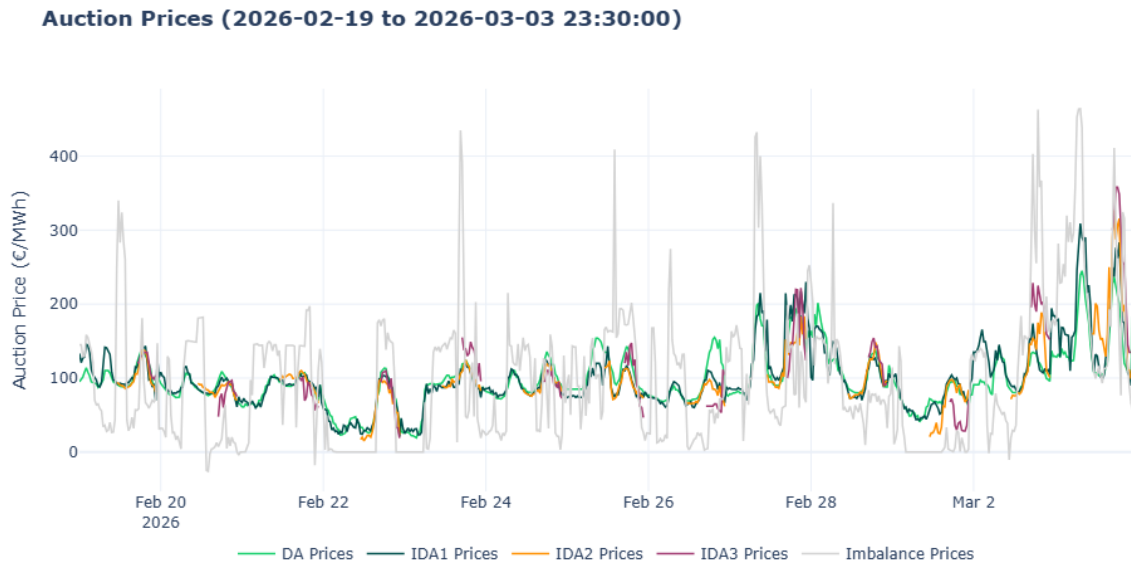


Figure 3: Comparison of Day-Ahead, Intraday (IDA1, IDA2, IDA3), and Imbalance market prices over the analysis period.

```
# Extract price data for all auction markets
df_prices = df[["PDA", "PIDA1", "PIDA2", "PIDA3", "PIMB"]]
```

Imbalance Price Volatility

This KPI measures the absolute percentage deviation of the imbalance price from the Day-Ahead price for each trading period. High volatility indicates periods where imbalance exposure carries significant financial risk, while low volatility suggests that forecast errors are less costly to settle. The average volatility across the analysis period is also calculated and displayed, providing a benchmark for assessing market conditions. This metric helps the desk identify patterns in price volatility and evaluate the risk associated with different trading strategies.

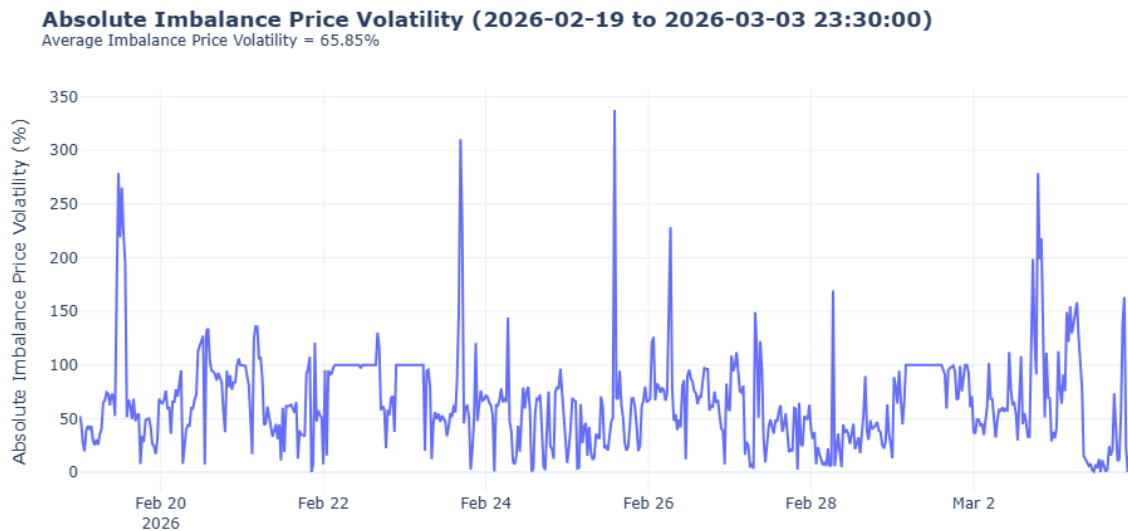


Figure 4: Absolute imbalance price volatility expressed as a percentage deviation from the Day-Ahead price.

```
# Calculate absolute percentage deviation of imbalance price from DA price
imbalance_price_pct = (abs(df_prices["PIMB"] - df_prices["PDA"]) /
↳ df_prices["PDA"]) * 100

# Calculate average volatility across the period
imbalance_price_pct_avg = imbalance_price_pct.mean()
```

Portfolio Positions vs Forecasts

This KPI tracks the evolution of trading positions across each auction stage (DA, IDA1, IDA2, IDA3, and IDC) and compares them against the DA and IDA3 generation forecasts. The position at each stage represents the cumulative volume sold up to that point. By visualising how positions build through successive auctions and comparing them to forecasts, traders can assess whether the desk is consistently underselling or overselling relative to expected generation. This is essential for understanding trading behaviour and identifying opportunities to optimise volume allocation across markets. When multiple asset groups are selected, the script also aggregates positions to show total portfolio exposure.

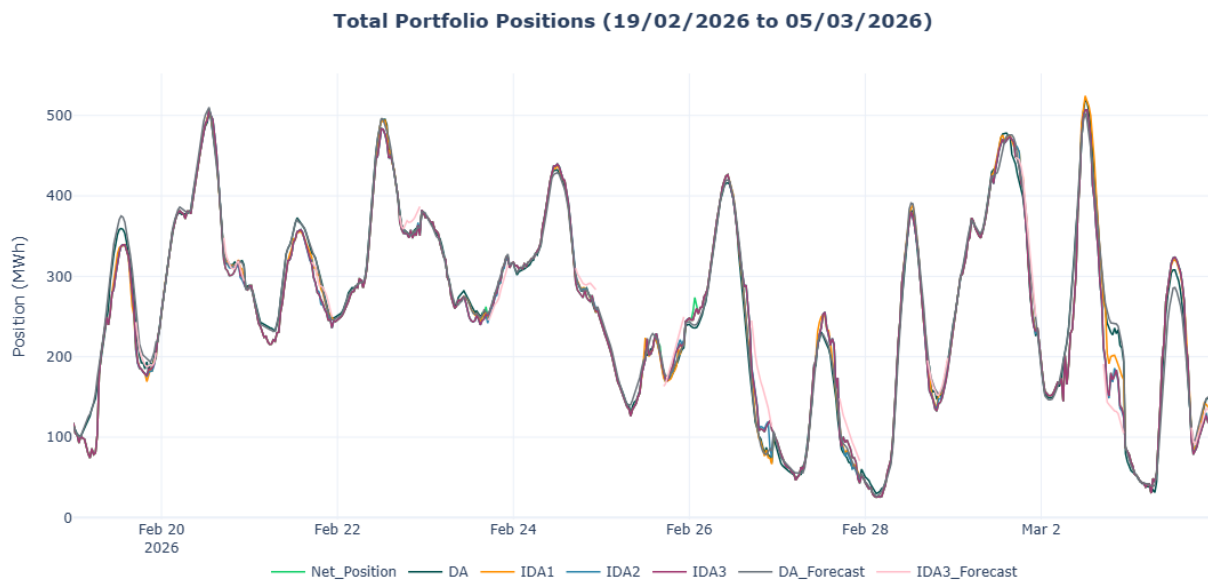


Figure 5: Portfolio positions at each auction stage (DA, IDA1, IDA2, IDA3) compared with DA and IDA3 generation forecasts.

```
# Calculate cumulative position at each auction stage
# Note: MW values are negative (sells), so negation gives positive position
group_DA_pos = -df["DA_MW"]
group_IDA1_pos = group_DA_pos - df["IDA1_MW"]
group_IDA2_pos = group_IDA1_pos - df["IDA2_MW"]
group_IDA3_pos = group_IDA2_pos - df["IDA3_MW"]

# Include IDC if available
try:
    group_IDC_pos = group_IDA3_pos - df["IDC_MW"]
except Exception:
    group_IDC_pos = group_IDA3_pos

# Net position is the total volume sold across all auctions
group_position = (-df["DA_MW"] - df["IDA1_MW"] - df["IDA2_MW"]
                  - df["IDA3_MW"] - df["IDC_MW"])

# Combine positions with forecasts for comparison
df_group_net_pos = pd.DataFrame({
    "Net_Position": group_position,
    "DA": group_DA_pos,
    "IDA1": group_IDA1_pos,
    "IDA2": group_IDA2_pos,
    "IDA3": group_IDA3_pos,
    "DA_Forecast": df["DA_Forecast"],
    "IDA3_Forecast": df["IDA3_Forecast"],
})

# For monthly aggregation, calculate mean positions
if agg_freq == "MS":
    df_group_net_pos = df_group_net_pos.resample(agg_freq).mean()
```

Dispatch Down Analysis

This KPI analyses the volume of generation that was dispatched down by the TSO, broken into two categories: curtailment (due to system-wide oversupply) and constraints (due to local network limitations). The plot displays metered generation alongside curtailed and constrained volumes as a stacked area chart, with the total available generation (what would have been produced without dispatch down instructions) shown as a reference line. This visualisation helps the desk understand how much potential revenue is being lost due to dispatch down events and identify patterns in when these events occur. The summary statistics show total volumes dispatched down and the number of periods affected.

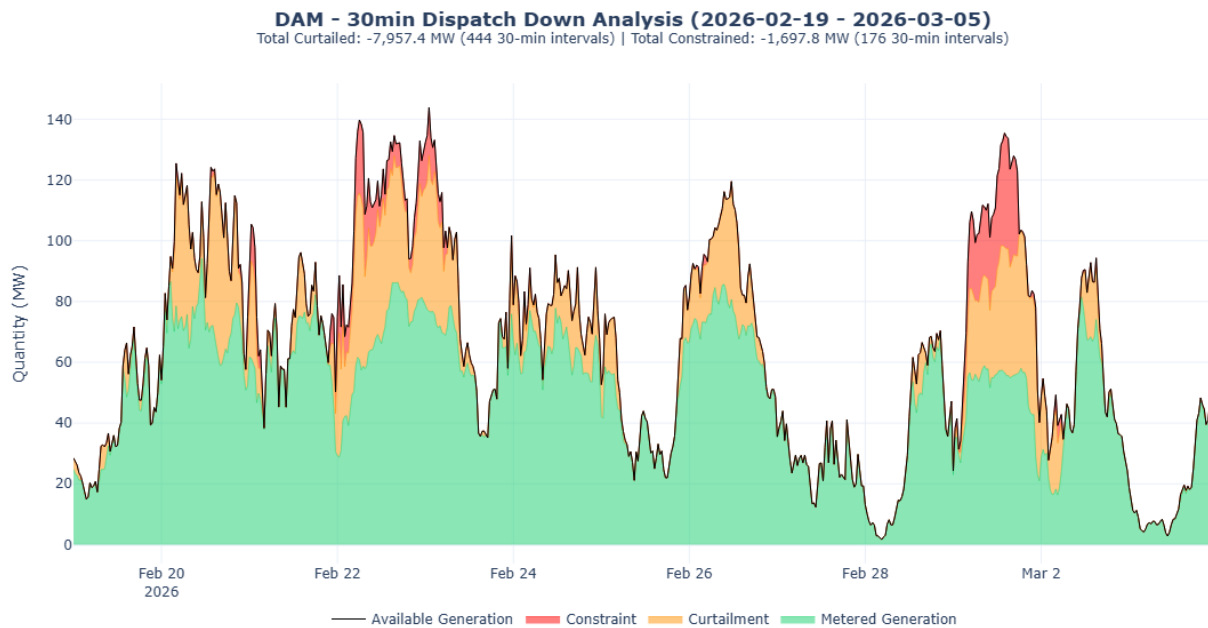


Figure 6: Dispatch down analysis showing metered generation, curtailment, and constraint volumes with available generation reference.

```
# Extract curtailment and constraint volumes from settlement data
# QAB = curtailment volume, QABCURL = constraint volume
curtailment = df["QAB"].fillna(0).resample(period).sum()
constraints = df["QABCURL"].fillna(0).resample(period).sum()

# Total dispatch down is the sum of both
total_dispatch_down = curtailment + constraints

# Metered generation (actual output)
qm = df["QM"].fillna(0).resample(period).sum()

# Available generation = what we would have produced without dispatch down
true_generation = qm - total_dispatch_down

# Summary statistics
total_curtailment = curtailment.sum()
total_constraints = constraints.sum()
periods_curtailed = (curtailment < 0).sum()
periods_constrained = (constraints < 0).sum()
```

Dispatch Down as Percentage of Available Generation

This KPI expresses curtailment and constraint volumes as a percentage of the total available generation, providing a normalised view of dispatch down impact. This is more meaningful than absolute volumes as it

accounts for variations in generation levels across different periods. The average percentages are calculated and displayed as horizontal reference lines, enabling quick identification of periods with above-average dispatch down. This metric is particularly useful for comparing dispatch down severity across different time periods or asset groups, and for understanding the proportional impact on portfolio revenue.

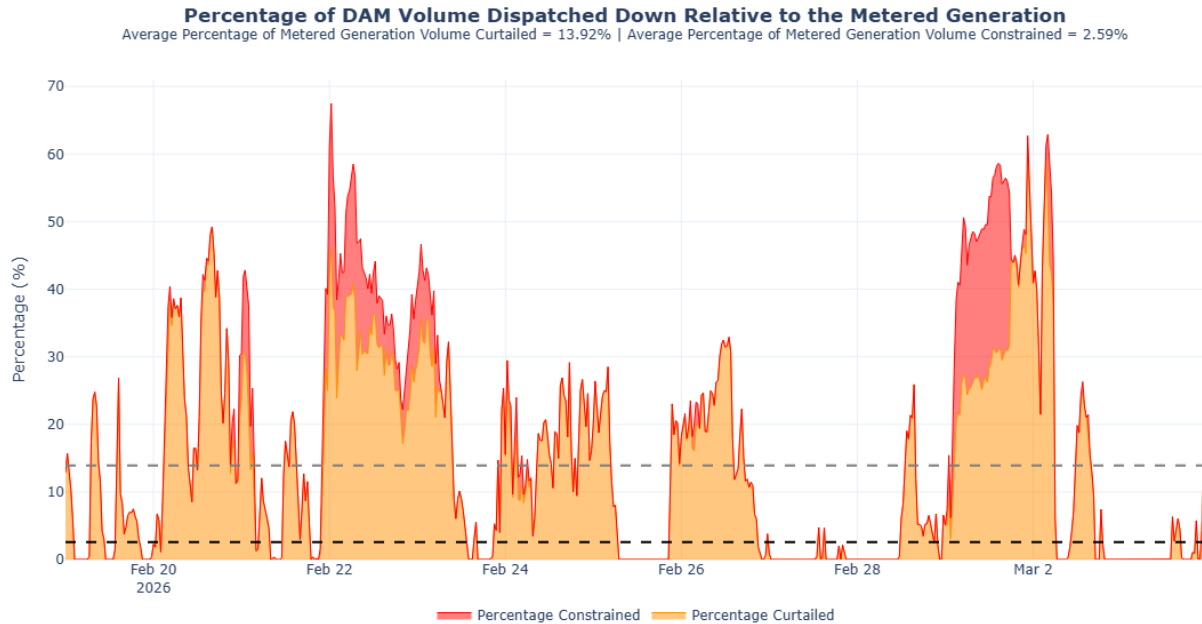


Figure 7: Percentage of available generation volume lost to curtailment and constraints, with average values indicated.

```
# Calculate percentage of available generation that was curtailed/constrained
# Note: curtailment values are negative, hence negation for positive
↳ percentage
percentage_curtailed = (-curtailment / true_generation) * 100
percentage_constrained = (-constraints / true_generation) * 100

# Calculate average percentages across the analysis period
average_percentage_curtailed = percentage_curtailed.mean()
average_percentage_constrained = percentage_constrained.mean()
```

Cumulative Long vs Short Balancing Costs

This KPI tracks the cumulative balancing costs incurred when the portfolio is in a long position (generated more than sold) versus a short position (generated less than sold). Understanding the split between long and short costs is critical for evaluating trading strategy effectiveness. If short costs dominate, it suggests the desk is consistently overselling relative to actual generation, exposing the portfolio to expensive buy-back in the balancing market. Conversely, if long costs dominate, the desk may be underselling and receiving unfavourable imbalance prices for excess generation. The percentage of periods spent long versus short is also calculated, providing insight into the overall trading bias. When multiple asset groups are analysed, the costs are aggregated to show total portfolio exposure. NB: The cost data in the below plots has been redacted to protect proprietary company information.

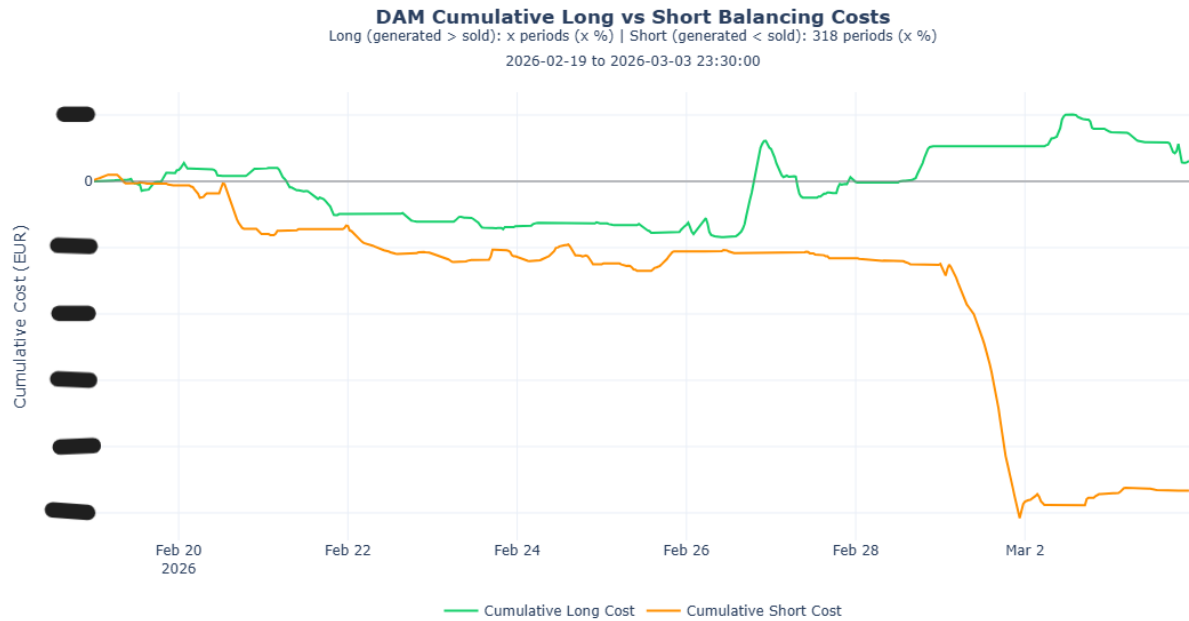


Figure 8: Cumulative balancing costs split by long (over-generated) and short (under-generated) positions.

```
# Determine if each period is long or short
# QEX = expected generation (sold position), QMLF = metered generation
# ↪ (loss-adjusted)
# Short: generated less than sold (QEX > QMLF)
# Long: generated more than sold (QEX < QMLF)
df["Short"] = 0
df["Long"] = 0
df.loc[(df["QEX"] - df["QMLF"]) > 0, "Short"] = 1
df.loc[(df["QEX"] - df["QMLF"]) < 0, "Long"] = 1

# Calculate costs attributed to each position type
df["Short_cost"] = df["Short"] * df["Cost"]
df["Long_cost"] = df["Long"] * df["Cost"]

# Cumulative costs over time
long_cumsum = df["Long_cost"].cumsum()
short_cumsum = df["Short_cost"].cumsum()

# Summary statistics
total_long_cost = df["Long_cost"].sum()
total_short_cost = df["Short_cost"].sum()
long_periods = df["Long"].sum()
short_periods = df["Short"].sum()
total_periods = len(df)

# Percentage of periods in each position
long_pct = (long_periods / total_periods * 100) if total_periods > 0 else 0
short_pct = (short_periods / total_periods * 100) if total_periods > 0 else 0
```

Trade All Auctions Simulation

This KPI simulates an alternative trading strategy where the desk adjusts its position in every intraday auction to match the latest available forecast, subject to market volume constraints. The simulation sells the DA

forecast at Day-Ahead, then trades the difference between each successive forecast update (IDA1, IDA2, IDA3) in the corresponding auction, limited to 30% of the available market volume. The resulting balancing costs are compared against the actual costs incurred by the desk's real trading decisions. This analysis quantifies whether more active participation in intraday auctions would have improved or worsened portfolio performance. A secondary chart shows the ratio of actual volume traded to simulated volume in each auction, highlighting where the desk may be under-utilising available liquidity.

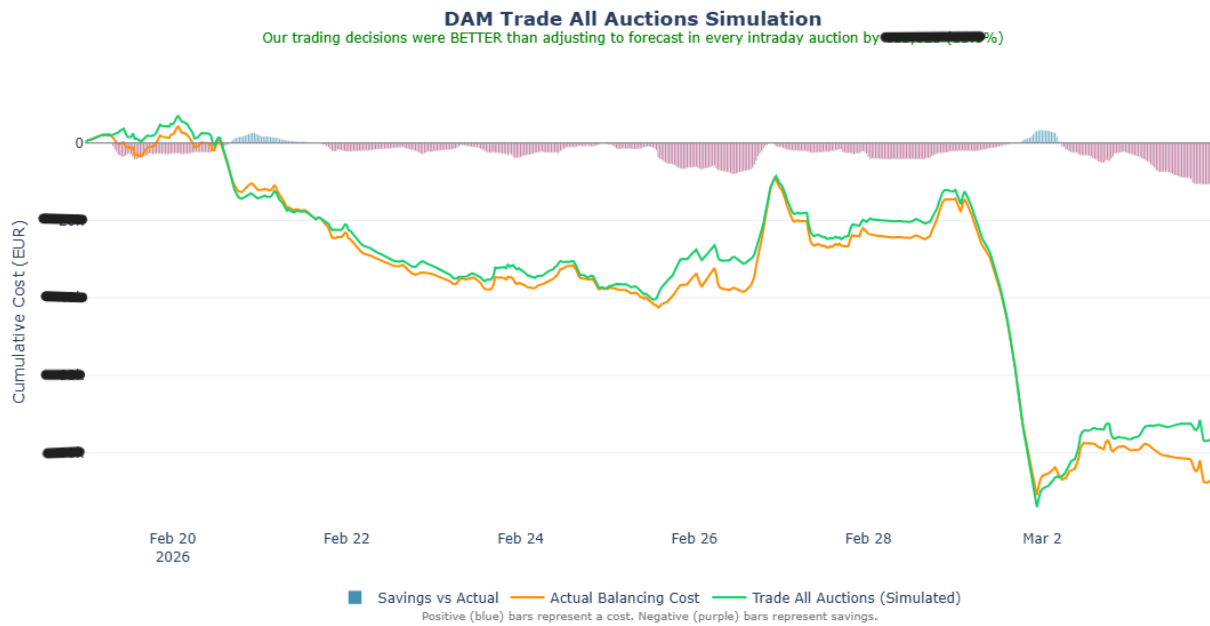


Figure 9: Comparison of actual cumulative balancing costs versus simulated costs from trading all auctions to forecast, with savings/losses indicated.

```
# Volume limit per auction (30% of available market volume)
auction_limit = 0.3
df_vol[f"{col}_lim"] = df_vol[col] * auction_limit

# DA: Sell entire DA forecast
df_t_all["DA_Trade"] = df2["DA_Forecast"]

# IDA1: Trade difference between IDA1 forecast and DA position
ida1_diff = df2["IDA1_Forecast"] - df_t_all["DA_Trade"]
df_t_all["IDA1_limit"] = np.minimum(abs(ida1_diff), df_vol["VIDA1_lim"])
df_t_all["IDA1_Trade"] = df_t_all["IDA1_limit"] * np.sign(ida1_diff)
df_t_all["IDA1_Pos"] = df_t_all["DA_Trade"] + df_t_all["IDA1_Trade"]

# IDA2: Trade difference between IDA2 forecast and IDA1 position
ida2_diff = df2["IDA2_Forecast"] - df_t_all["IDA1_Pos"]
df_t_all["IDA2_limit"] = np.minimum(abs(ida2_diff), df_vol["VIDA2_lim"])
df_t_all["IDA2_Trade"] = df_t_all["IDA2_limit"] * np.sign(ida2_diff)
df_t_all["IDA2_Pos"] = df_t_all["IDA1_Pos"] + df_t_all["IDA2_Trade"]

# IDA3: Trade difference between IDA3 forecast and IDA2 position
ida3_diff = df2["IDA3_Forecast"] - df_t_all["IDA2_Pos"]
df_t_all["IDA3_limit"] = np.minimum(abs(ida3_diff), df_vol["VIDA3_lim"])
df_t_all["IDA3_Trade"] = df_t_all["IDA3_limit"] * np.sign(ida3_diff)
df_t_all["IDA3_Pos"] = df_t_all["IDA2_Pos"] + df_t_all["IDA3_Trade"]
```

```

# Calculate income from each auction
df_t_all["DA_income"] = df_t_all["DA_Trade"] * df2["PDA"] / 2
df_t_all["IDA1_income"] = df_t_all["IDA1_Trade"] * df2["PIDA1"] / 2
df_t_all["IDA2_income"] = df_t_all["IDA2_Trade"] * df2["PIDA2"] / 2
df_t_all["IDA3_income"] = df_t_all["IDA3_Trade"] * df2["PIDA3"] / 2

# Total auction income
df_t_all["EA_Income"] = (df_t_all["DA_income"] + df_t_all["IDA1_income"]
                        + df_t_all["IDA2_income"] + df_t_all["IDA3_income"])

# Imbalance volume and cost
df_t_all["Trade_all_QIMB"] = df2["QMLF"] - df_t_all["IDA3_Pos"] / 2
df_t_all["Trade_all_imb"] = df_t_all["Trade_all_QIMB"] * df2["PIMB"]

# Simulated balancing cost
df_t_all["Trade_all_Bal_Cost"] = df2["Client_Payment"] -
↳ (df_t_all["EA_Income"] + df_t_all["Trade_all_imb"])

# Compare actual vs simulated
total_actual = df2[cost].sum()
total_simulated = df_t_all["Trade_all_Bal_Cost"].sum()
total_savings = total_actual - total_simulated # Positive = simulation saved
↳ money

# Volume comparison: actual vs simulated per auction
vols.loc["Ratio", auction] = (vols.loc["Actual", auction]
                              / vols.loc["Simulated", auction])

```

Imbalance Position Analysis

This KPI visualises the imbalance position for each trading period, calculated as the difference between the expected (sold) volume and the actual metered generation. A positive position indicates the portfolio was short (generated less than sold), while a negative position indicates the portfolio was long (generated more than sold). The chart displays the proportion of periods spent in each state and the average position across the analysis period. For longer timeframes, the data is aggregated to monthly means to show trends more clearly. Understanding the distribution and magnitude of imbalance positions helps identify systematic biases in trading behaviour — for example, if the portfolio is consistently short, it may indicate a tendency to oversell relative to actual generation capability.

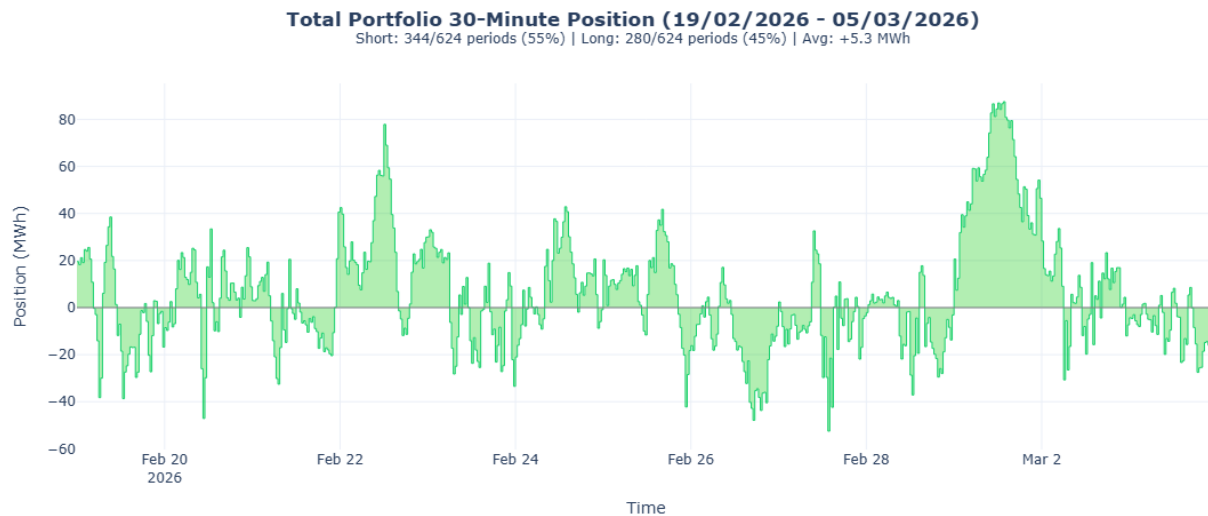


Figure 10: Imbalance position over time showing periods where the portfolio was short (positive) versus long (negative).

```
# Calculate imbalance position for each 30-minute period
# QEX = expected generation (sold position)
# QMLF = metered generation (loss-adjusted)
# Positive = short (sold more than generated)
# Negative = long (generated more than sold)
position_30min = df["QEX"] - df["QMLF"]

# For monthly aggregation, calculate mean position per month
position_monthly = position_30min.resample("MS").mean()

# Summary statistics
avg_position = position_30min.mean()
periods_short = (position_30min > 0).sum() # Sold more than generated
periods_long = (position_30min < 0).sum() # Generated more than sold
total_periods = len(position_30min)

# Percentage of periods in each state
short_pct = 100 * periods_short / total_periods
long_pct = 100 * periods_long / total_periods

# For portfolio aggregation across multiple asset groups
portfolio_position_30min = portfolio_position_30min.add(position_30min,
↳ fill_value=0)
```

3.2 Solar Imbalance Price Trading Strategy

An issue that the team was facing was to do with solar assets and the optimal allocation of forecast volume between the Day-Ahead (DA) and Intraday (IDA) markets. Currently, the desk sells 100% of the DA forecast in the Day-Ahead auction. However, solar forecasts can be unreliable due to cloud cover variability, leading to significant imbalance exposure when actual generation differs from the forecast position. This has resulted in substantial margin losses due to solar balancing costs.

I was tasked with investigating whether alternative strategies could reduce imbalance costs and improve overall revenue, despite the trade-off that intraday prices tend to be slightly lower than Day-Ahead prices on average.

Strategy Comparison

I developed a Python script to backtest the following strategies using historical data:

- **100% DA Strategy (Baseline):** Sell the entire DA forecast volume at the Day-Ahead price. Any difference between actual generation and the forecast is settled at the imbalance price.
- **80% DA + IDA1 Strategy:** Sell 80% of the DA forecast at the Day-Ahead price. Then, in the IDA1 auction, sell additional volume up to the IDA1 forecast (which is typically more accurate as it is closer to delivery). The remaining imbalance is settled at the imbalance price. The trade-off here is that IDA1 prices tend to be slightly lower than DA prices on average.
- **80% DA + IDA2 Strategy (In Development):** Sell 80% of the DA forecast at the Day-Ahead price, but only for the IDA2 auction periods (11:00 to 23:00). Then update the position at IDA2, which is even closer to real-time for those delivery periods. This strategy aims to capture improved forecast accuracy closer to delivery, though IDA2 prices tend to be more volatile which may offset the benefits.

The script calculates revenue from each market and the resulting imbalance costs to determine total revenue for each strategy. The key calculation compares:

$$\text{Total Revenue} = \text{DA Revenue} + \text{IDA Revenue} + \text{Imbalance Settlement} \quad (4)$$

where the imbalance settlement can be positive (if over-generating) or negative (if under-generating relative to the sold position). These auction price differences can be seen in Figure ??.

Current Status

The initial backtesting framework for the 80% DA + IDA1 strategy has been completed and is functional. Preliminary results are being analysed to determine whether the improved forecast accuracy at IDA1 outweighs the slightly lower auction prices. The second strategy involving IDA2 adjustments for afternoon/evening periods is currently in development.

```
1 # Strategy 1 comparison code extract
2 grp = "SOL"
3 df_grp = df[df["Group"] == grp]
4
5 # Volumes and Prices
6 DA_volume = df_grp["DA_Forecast"]
7 DA_volume_80pct = DA_volume * 0.8
8 IDA1_volume = df_grp["IDA1_Forecast"]
9 PDA, PIDA1, PIMB = df_grp["PDA"], df_grp["PIDA1"], df_grp["PIMB"]
10 actual_volume = df_grp["QM"]
11
12 # 100% DA Strategy
13 DA_revenue = DA_volume * PDA
14 DA_imbalance_cost = (actual_volume - DA_volume) * PIMB
15 DA_total_revenue = DA_revenue.sum() + DA_imbalance_cost.sum()
16
17 # 80% DA + IDA1 Strategy
18 s1_da_revenue = DA_volume_80pct * PDA
19 s1_ida1_volume = (IDA1_volume - DA_volume_80pct).clip(lower=0)
20 s1_ida1_revenue = s1_ida1_volume * PIDA1
21 s1_total_sold = DA_volume_80pct + s1_ida1_volume
22 s1_imbalance_cost = (actual_volume - s1_total_sold) * PIMB
23 s1_total_revenue = s1_da_revenue.sum() + s1_ida1_revenue.sum() +
   ↪ s1_imbalance_cost.sum()
```

Preliminary Results

The backtesting results for the 80% DA + IDA1 strategy show mixed performance depending on the timeframe analysed, with neither approach consistently outperforming the other. Several factors likely contribute to this variability:

- **Forecast accuracy variation:** The improvement in forecast accuracy between DA and IDA1 is not consistent across all days. On days with stable cloud cover, the DA forecast may already be sufficiently accurate, making the IDA1 adjustment unnecessary.
- **Price spread dynamics:** The difference between DA and IDA1 prices varies significantly. When IDA1 prices are substantially lower than DA, the revenue lost from selling at IDA1 may outweigh the savings from reduced imbalance costs.
- **Imbalance price volatility:** The imbalance price can be highly volatile. In periods where imbalance prices are close to DA prices, the penalty for forecast errors is lower, reducing the benefit of improved accuracy.
- **Seasonal effects:** Solar generation patterns and forecast accuracy likely vary by season, with summer months potentially showing different results compared to winter due to longer daylight hours and differing weather patterns.

Further analysis is required to identify the specific market conditions under which each strategy performs best. This could enable a conditional approach where the trading strategy is adapted based on forecast confidence, expected price spreads, or other market indicators.

3.3 Paper Trading

As an intern, I will not be executing live trading strategies on the desk. Instead, for the remainder of my internship I will be undertaking ‘paper’ trading. This process allows me to formulate trading strategies and execute simulated trades in the I-SEM Day-Ahead Market using the same live data that the traders use. A Python script tracks my performance across each trading day, enabling direct comparison against desk performance. My goal is to develop my trading intuition and decision-making skills, ultimately aiming to perform as well as, or even outperform, the desk’s actual results.

Strategy Development Framework

To support my paper trading activities, I developed a comprehensive strategy guide document outlining the key factors that influence trading decisions in the I-SEM market. This framework helps structure the decision-making process by analysing each variable in terms of its impact on generation, prices, and system position. The main factors considered include:

- **Weather conditions:** Temperature, wind speeds, cloud cover, precipitation, and atmospheric pressure all directly influence both renewable generation output and electricity demand.
- **System indicators:** SNSP levels, curtailment risk, interconnector flows, and Net Imbalance Volume (NIV) provide insight into the likely system position and price dynamics.
- **Forecast analysis:** Comparing forecasts from multiple providers (EirGrid, Meteo, HRES) and tracking revisions between DA and ID timeframes helps assess confidence levels.
- **External factors:** Gas prices, planned outages, day of the week, and time of day all contribute to the overall market picture.

Underselling vs Overselling

A core concept in the strategy framework is the decision to undersell or oversell relative to the forecast:

- **Underselling** (selling less than forecast) is appropriate when forecast uncertainty is high, when the system is expected to be long (over-generating), or when imbalance prices are expected to exceed DA prices.
- **Overselling** (selling more than forecast) is appropriate when forecasts are conservative, when DA prices are high relative to expected imbalance prices, or when the system is expected to be short.

The key principle is to position on the opposite side of the system — being long when the system is short (receiving high imbalance prices for excess generation) or being short when the system is long (buying back shortfalls at low imbalance prices).

Decision Matrix Approach

To systematically evaluate trading decisions, I developed a decision matrix that maps out each influencing factor and its corresponding trading implication. For example, during the morning ramp-up period when multiple variables are changing simultaneously, the matrix helps aggregate individual factor assessments into an overall trading decision. This structured approach reduces the risk of overlooking important signals and provides a documented rationale for each trade.

Learning Objectives

Through paper trading, I aim to:

- Develop practical experience interpreting weather forecasts and market data in real-time.
- Build intuition for how different market conditions affect optimal trading strategies.
- Learn from mistakes in a risk-free environment before any potential future involvement in live trading.
- Validate the theoretical framework developed in the strategy guide against actual market outcomes.

3.4 Weekend Trading Look-Aheads

One of my regular responsibilities has been preparing weekend trading look-ahead reports. These reports are produced on Friday afternoons and distributed to the weekend trading team, providing a consolidated analysis of expected weather conditions, demand patterns, and generation forecasts for the upcoming weekend. The purpose is to equip traders with the information needed to make informed decisions when the full weekday team is not available, highlighting key risks and opportunities that may arise.

Each report synthesises data from multiple forecast providers including EirGrid, Meteo, and MetDesk, analysing factors such as system demand, wind and solar generation, temperature, precipitation, and cloud cover. The analysis identifies periods of forecast uncertainty, potential price volatility, and any weather events that could significantly impact portfolio performance.

Below is an example of a weekend look-ahead report that I produced for the weekend of 20th–21st February 2026.

20/02/2026 Weekend Look-Ahead

Demand Analysis:

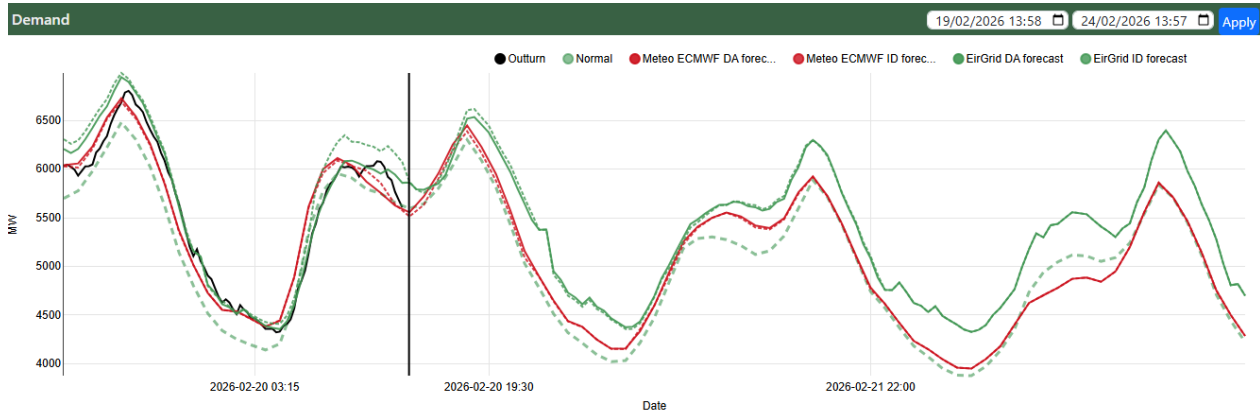


Figure 11: System demand forecasts from EirGrid and Meteo alongside actual outturn.

- Peak demand levels are similar across both days: Saturday (6.3 GW EirGrid vs 5.92 GW Meteo) and Sunday (6.4 GW EirGrid vs 5.86 GW Meteo).
- EirGrid forecasts consistently predict higher demand than Meteo, suggesting a degree of forecast divergence that should be monitored.
- Meteo predicts higher midday demand on Saturday compared to Sunday, which may influence solar generation expectations during peak generation hours.

Wind Conditions:

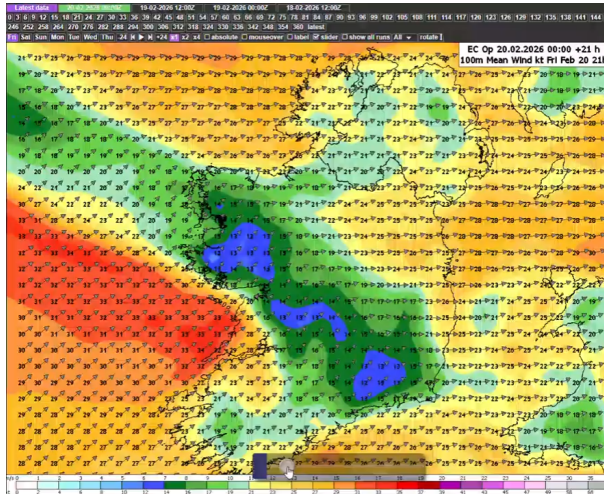


Figure 12: MetDesk wind speed forecast.

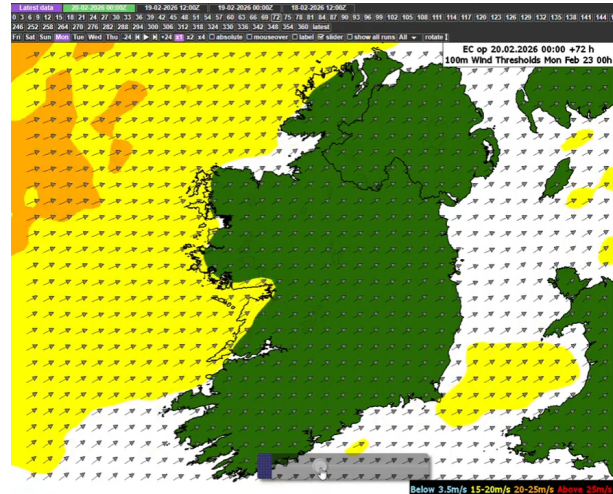


Figure 13: MetDesk wind cut-out risk forecast.

- Wind speeds are expected to remain consistently moderate throughout the weekend, providing stable generation conditions.
- An Atlantic system approaching from the south-west will bring elevated wind speeds to western coastal areas.
- Wind speeds in the west are forecast to increase around 15:00 on Sunday, which may boost generation towards the end of the weekend.

- No significant risk of cut-in or cut-out conditions is anticipated, allowing confident trading at Day-Ahead.

Wind Generation Forecasts:



Figure 14: Portfolio wind generation forecast with confidence bands.

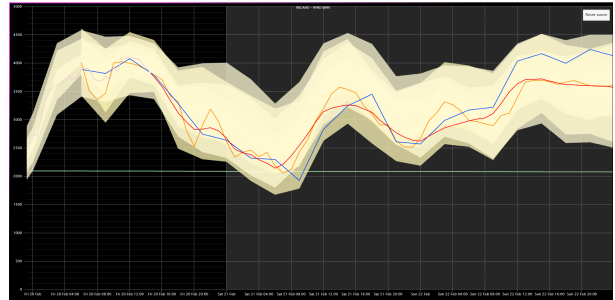


Figure 15: MetDesk system-wide wind generation forecast.

- Portfolio wind generation confidence bands widen around midday on Saturday, indicating increased forecast uncertainty during this period. Conservative positioning may be advisable.
- System-wide wind generation shows a widening forecast plume as the weekend progresses, with greater deviation to the upside suggesting potential for higher-than-forecast generation.
- The weekend trend shows a gradual ramp from approximately 2 GW at 08:00 on Saturday to 3.5–4 GW by 20:00 on Sunday, implying progressively lower prices as wind penetration increases.

Temperature:

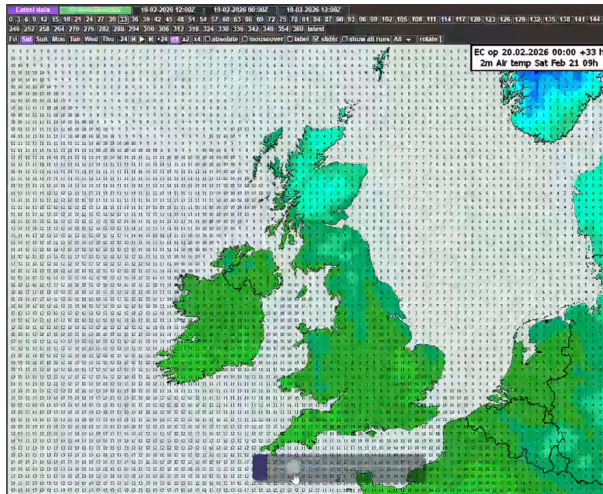


Figure 16: Temperature forecast for Ireland and UK.

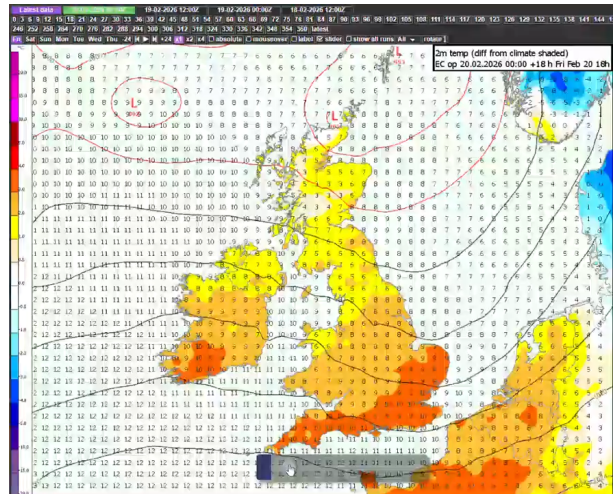


Figure 17: Temperature anomaly forecast.

- Temperatures are forecast to range between 7–13°C across Ireland and the UK throughout the weekend.
- Conditions are 2–5°C above seasonal norms, which is likely to suppress heating-driven demand and contribute to lower overall system demand compared to typical February levels.

Precipitation and Cloud Cover:

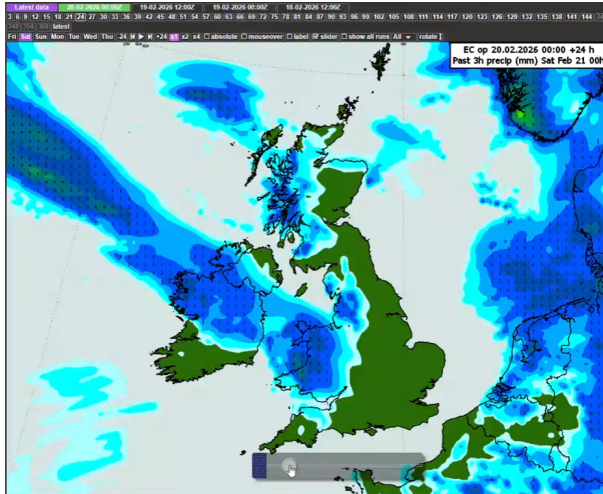


Figure 18: Precipitation forecast.

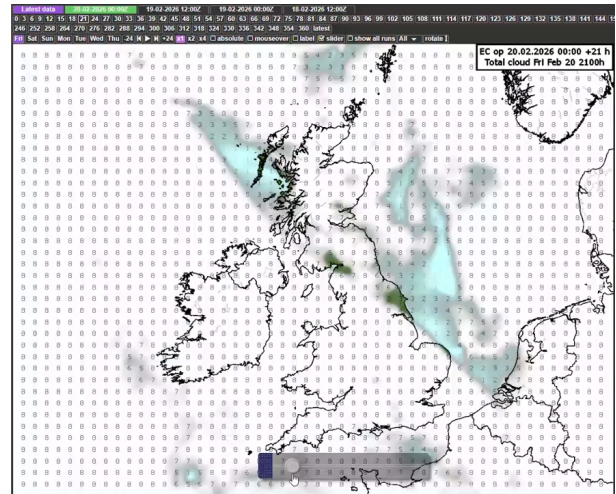


Figure 19: Cloud cover forecast.

- Moderate rainfall is expected across Ireland and the UK, with a heavier band arriving on Saturday around 15:00.
- Saturday will experience heavier and more persistent rainfall compared to Sunday.
- Cloud cover will be extensive throughout the weekend; however, clearing is expected around midday on Sunday, which should improve solar generation conditions during peak production hours.

Solar Generation:

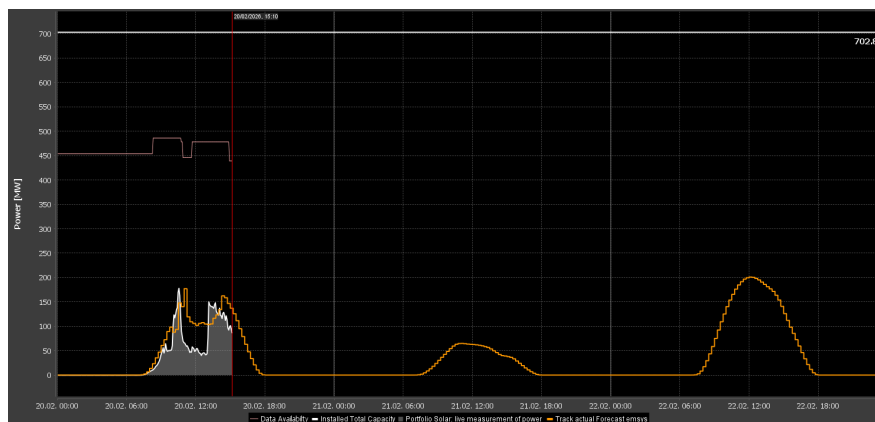


Figure 20: Portfolio solar generation forecast.

- Solar generation is forecast to be higher on Sunday, correlating with the expected reduction in cloud cover around midday.
- Saturday's solar output is expected to be relatively flat and suppressed due to persistent cloud cover and rainfall.
- Trading strategy should account for the asymmetry between the two days, with more conservative solar positions on Saturday.

Summary and Key Risks:

- **Wind:** Stable and moderate conditions with no cut-out risk. Increasing generation towards Sunday evening may push prices lower.

- **Solar:** Significant difference between Saturday (low, flat) and Sunday (higher, variable). Sunday midday presents the best solar opportunity.
- **Demand:** Above-average temperatures will likely suppress demand. Forecast divergence between EirGrid and Meteo should be monitored.
- **Overall:** A relatively benign weekend with no major weather risks. The system is likely to trend long as wind increases, suggesting underselling may be appropriate towards Sunday evening.

3.5 Data Scientist in Python

I have been working through the Dataquest *Data Scientist in Python* Course. This course takes people from having a base knowledge in Python, to being able to apply machine learning techniques to solve complex problems. So far, I have completed the *Python Introduction* and *Data Analysis and Visualisation* sections of the course. I am currently in the middle of completing the *Data Cleaning* section.

I have definitely brushed up on and developed more of my Python skills massively since starting my internship. I have become proficient at cleaning and manipulating dataframes using Pandas, performing statistical analysis using NumPy, and producing visually appealing graphs in plots through Matplotlib, Plotly and Seaborn.



Figure 21: Python Introduction

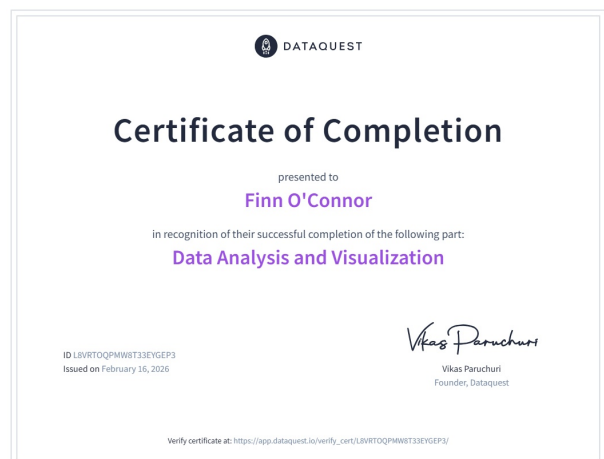


Figure 22: Data Analysis and Visualisation

The course is quite long and dives into complex topics, but I'm currently just 25% of the way through. I am excited to continue learning and developing skills so I can hopefully apply them to my Masters thesis project next year. The remaining sections of the course include:

Data Cleaning, The Command Line, Working with Data Sources Using SQL, APIs and Web Scraping in Python, Probability and Statistics, Machine Learning in Python, Deep Learning in Python, and Advanced Topics in Data Science.

4 Conclusions

So far, this internship has provided practical experience in renewable energy trading, combining operational responsibilities with quantitative analysis. Working with the Shift Trading Team has given me exposure to the full trading workflow—from forecast analysis and trade execution—reinforcing how theoretical market concepts translate into commercial decisions.

My primary technical contribution has been the development of a consolidated KPI analysis script that enables the trading desk to evaluate performance across multiple asset groups and timeframes. Building this tool required integrating data from various sources, implementing financial calculations, and creating clear visualisations. The process strengthened my proficiency in Python, SQL, and data analysis, while also developing my understanding of electricity market mechanics and the challenges of trading variable renewable generation.

5 Plan for Future Work

The remaining months of the internship will focus on several key areas. First, I will continue refining the KPI script based on feedback from the trading desk, including adding new metrics, improving visualisation clarity, and further optimising query performance. The data engineering team is currently developing a more robust data retrieval system, which will be integrated into the script once available.

Second, I plan to apply the developed tools to conduct deeper analysis of trading performance, examining patterns across different market conditions and identifying potential strategy improvements. This may involve applying statistical models to analyse the relationship between forecasts and generation/outcome, or evaluate trading performance.

Third, I aim to perform well in paper trading exercises, applying the market knowledge I have gained to make strategised trading decisions. This will help bridge the gap between analytical work and real-time trading, developing my intuition for market dynamics and decision-making under uncertainty.

Fourth, I plan to complete the Energy Traders Europe: Energy Market Fundamentals course, which will deepen my understanding of European electricity and gas markets, and the Data Scientist in Python course on Dataquest, which will further develop my programming and data analysis skills. Both courses complement the practical experience gained during the internship.

Finally, I will prepare documentation for the tools I have developed to ensure they can be maintained and extended after the internship concludes. This includes user guides explaining how to run the scripts and interpret outputs, as well as technical documentation covering the code structure and data dependencies. I will also continue supporting the team with any additional tasks or projects that arise, whether analytical, operational, or administrative, broadening my exposure to different aspects of the company.